

CONTENTS

Práctico 5 y Práctico 6 (Algoritmos Ávidos) — Programación 3

 FING UdelaR · Resolución completa y verificada

Práctico 5

 Ejercicio 1 (K&T Ex. 4.5) — Estaciones de cobertura en una carretera

 (a) Algoritmo

 (b) Corrección — greedy stays ahead

 Ejercicio 2 (K&T Ex. 4.6) — Triatlón: orden de largada

 (a) Algoritmo

 (b) Corrección — argumento de intercambio

 Ejercicio 3 (K&T Ex. 4.18) — Camino más rápido con tiempos de viaje variables

 (a) Algoritmo — variante de Dijkstra

 (b) Corrección

 (c) Complejidad $O(m \log n)$

Práctico 6

 Ejercicio 1 (K&T Ex. 4.8) — Unicidad del MST con costos distintos

 Ejercicio 2 (K&T Ex. 4.21) — MST de un “casi árbol” en tiempo $O(n)$

 (a) Algoritmo

 (b) Corrección

 (c) Complejidad $O(n)$

Ejercicio 3 (K&T Ex. 4.10) — MST tras agregar una arista

(a) Mostrar que T' es un MST de G'

(b) Algoritmo $O(n)$ para construir el MST de G' a partir de G , T y e

Resumen de técnicas usadas en estos prácticos

Práctico 5 y Práctico 6 (Algoritmos Ávidos)

— Programación 3

FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

⚠ **Nota general sobre el material fuente:** para resolver estos prácticos se usó como referencia “Semana 5 (letra y resolución).pdf” y “Semana 6 (Resuelto MB).pdf” — documentos de cátedra de años anteriores (2018) que resuelven los mismos ejercicios del libro de Kleinberg & Tardos, aunque agrupados en un orden distinto al de los Prácticos 5 y 6 de 2024. Cada solución de este documento fue **verificada de forma independiente** (no transcripta) contra el enunciado y contra el esquema formal de demostración correspondiente (greedy stays ahead / exchange argument). Se marcan con ⚠ los puntos donde el material “oficial” tiene errores, imprecisiones o pasos poco claros.

Correspondencia de ejercicios (los prácticos de 2024 reordenan los mismos ejercicios de K&T que los documentos de “Semana”):

Práctico 2024	Ejercicio K&T	Resuelto en
Práctico 5, Ej.1	Ex. 4.5 (estaciones de cobertura)	Semana 5, Ej.1
Práctico 5, Ej.2	Ex. 4.6 (triatlón)	Semana 5, Ej.2
Práctico 5, Ej.3	Ex. 4.18 (viaje con tiempos variables)	Semana 6, Ej.3
Práctico 6, Ej.1	Ex. 4.8 (unicidad del MST)	Semana 6, Ej.1
Práctico 6, Ej.2	Ex. 4.21 (casi árbol)	Semana 6, Ej.2
Práctico 6, Ej.3	Ex. 4.10 (MST tras agregar una arista)	Semana 5, Ej.3

Práctico 5

Ejercicio 1 (K&T Ex. 4.5) — Estaciones de cobertura en una carretera

Enunciado. Dadas las posiciones de n casas sobre una carretera recta, ubicar la menor cantidad posible de estaciones de manera que ninguna casa esté a más de 4 km de una estación.

(A) ALGORITMO

Sea $C = \{c_1 \leq c_2 \leq \dots \leq c_n\}$ las posiciones de las casas, **ordenadas de menor a mayor**.

```

UbicarEstaciones(C):
    ordenar C de menor a mayor
    E = ∅
    mientras C no esté vacío:
        c = menor elemento de C (la casa sin cubrir más a la izquierda)
        e = c + 4                                # ubico la estación lo más lejos posible
        agregar e a E                             # que aún cubra a c
        quitar de C todas las casas c' tales que c' ≤ e (equivalente a c' ≤ c+4,
                                                    y por estar ordenadas, c' ≤
c+8
                                                    respecto de la estación an-
terior no aplica:
                                                    basta c' ≤ e)
    devolver E
  
```

Complejidad: ordenar es $O(n \log n)$; el resto del algoritmo recorre cada casa una vez (con dos punteros, sin necesidad de “buscar” en cada iteración) — $O(n)$. Total **$O(n \log n)$** .

(B) CORRECCIÓN — GREEDY STAYS AHEAD

⚠ **Corrección de un error en la resolución “oficial” (Semana 5, Ej.1(b)).** El documento de cátedra enuncia correctamente el **Lema Stay Ahead** como “para todo $1 \leq k \leq n$, $e_k \geq e_k^*$ ” (la k -ésima estación ávida está ubicada tan lejos o más lejos que la k -ésima estación óptima), y el caso base también concluye correctamente $e_1 \geq e_1^*$. Pero en el **paso inductivo**, después de fijar como tesis a demostrar “ $e_{k+1} \geq e_{k+1}^*$ ”, el texto termina la demostración escribiendo la conclusión con el signo invertido: “*entonces necesariamente $e_{k+1} \leq e_{k+1}^*$* ”. Esto es un error de tipeo/signo que contradice el enunciado del propio lema dos líneas antes — la desigualdad correcta, consistente con el resto de la demostración (y con la que efectivamente hace falta para el absurdo final), es $e_{k+1} \geq e_{k+1}^*$. A continuación se rehace la demostración completa con el signo correcto.

Sea $E = \{e_1, e_2, \dots, e_n\}$ la solución que produce el algoritmo (indexada en el orden en que se agregan, de izquierda a derecha) y sea $E^* = \{e_1^*, \dots, e_m^*\}$ una solución óptima **arbitraria** (también ordenada de izquierda a derecha). Queremos probar $n = m$.

Lema (greedy se mantiene adelante). Para todo k tal que $1 \leq k \leq \min(n, m)$: $e_k \geq e_k^*$.

Demostración por inducción en k .

- **Caso base ($k=1$):** $e_1 = c_1 + 4$, donde c_1 es la casa más a la izquierda de todas. Como cualquier estación que cubra a c_1 debe estar en el rango $[c_1-4, c_1+4]$ (o más precisamente, para que exista una estación que cubra a c_1 situada a la izquierda de todo, en la práctica $e_1^* \leq c_1+4$, ya que ubicar la primera estación óptima más allá de c_1+4 dejaría a c_1 sin cobertura), se cumple $e_1^* \leq c_1+4 = e_1$. ✓
- **Paso inductivo:** hipótesis $e_k \geq e_k^*$. Sea c la casa sin cubrir más a la izquierda **en el momento en que el algoritmo ávido agrega la estación e_{k+1}** (es decir, la primera casa con posición $> e_k$). Como $e_k \geq e_k^*$, toda casa cubierta por las primeras k estaciones óptimas (posición $\leq e_k^*$) también está cubierta por las primeras k estaciones ávidas (posición $\leq e_k$, y $e_k \geq e_k^*$), así que c tampoco está cubierta por $\{e_1^*, \dots, e_k^*\}$. Por lo tanto, la estación e_{k+1}^* (si existe) debe cubrir a c , lo que exige $e_{k+1}^* \geq c - 4$, o dicho de otro modo, $c \leq e_{k+1}^* + 4$ — y como cualquier estación que cubra a c satisface estación $\leq c+4$, en particular $e_{k+1}^* \leq c + 4$. El algoritmo ávido, por su parte, define $e_{k+1} = c + 4$ exactamente (la mayor posición posible que aún cubre a c). Luego $e_{k+1} = c+4 \geq e_{k+1}^*$. ■ (Ahora el signo es consistente con el lema enunciado.)

Optimalidad (por absurdo). Supongamos $n > m$. Consideremos las primeras m estaciones ávidas $\{e_1, \dots, e_m\}$. Por el lema, $e_m \geq e_m^*$. Como el algoritmo ávido no se detuvo en el paso m (siguió hasta $n > m$), existe al menos una casa c no cubierta por $\{e_1, \dots, e_m\}$ — es decir, $c > e_m$. Pero entonces $c > e_m \geq e_m^*$, así que c tampoco está cubierta por $\{e_1^*, \dots, e_m^*\}$. Esto contradice que E^* (con sus m estaciones) sea una solución **factible** (cubre a todas las casas). Por lo tanto $n \leq m$. Y como E^* es óptima (tiene la mínima cantidad posible) y E es factible, $m \leq n$. Luego **$n = m$: E es óptima.** ■

Ejemplo numérico de verificación. Casas en posiciones $\{0, 1, 3, 7, 8, 12, 15, 20\}$ km.

- Estación 1: casa más a la izquierda = 0 $\rightarrow e_1 = 4$. Cubre $[0, 4]$: quita 0, 1, 3.
- Restan $\{7, 8, 12, 15, 20\}$. Casa más a la izquierda = 7 $\rightarrow e_2 = 11$. Cubre $[7, 11]$: quita 7, 8.
- Restan $\{12, 15, 20\}$. Casa más a la izquierda = 12 $\rightarrow e_3 = 16$. Cubre $[12, 16]$: quita 12, 15.
- Restan $\{20\}$. Casa más a la izquierda = 20 $\rightarrow e_4 = 24$. Cubre $[20, 24]$: quita 20.

Total: **4 estaciones** en {4, 11, 16, 24}. Se verifica a mano que con 3 estaciones es imposible cubrir las 8 casas (el rango total es 0 a 20 = 20 km, y cada estación cubre a lo sumo 8 km de rango; con los huecos reales entre casas consecutivas (0→7 hay 7km, 8→12 hay 4km, 12→20 hay 8km) 3 estaciones no alcanzan y 4 sí son necesarias y suficientes).

Ejercicio 2 (K&T Ex. 4.6) — Triatlón: orden de largada

Enunciado. n competidores, cada uno con tiempos de natación s_i , ciclismo b_i y carrera r_i . La piscina (natación) solo admite un nadador a la vez, así que la largada define un orden $i_1, i_2, \dots, i_n$ en el que entran a nadar, uno a la vez; las etapas de ciclismo y carrera se pueden solapar entre competidores distintos. Minimizar el instante en que termina el último competidor.

 **Nota sobre el material fuente (Semana 5, Ej.2).** La resolución oficial da la idea correcta en el párrafo de “Solución” (ordenar por $b_i + r_i$ decreciente y usar argumento de intercambio), pero dentro del documento los apartados “(a)” y “(b)” quedan **vacíos** (solo aparecen los encabezados “(a)” y “(b)” sin contenido debajo) — es decir, la respuesta completa está en el párrafo previo sin la separación formal pedida por el enunciado. Además, la frase que describe el intercambio (“ i consigue salir afuera de la piscina cuando j estuvo previamente fuera de ella...”) es confusa y de difícil lectura tal como está redactada. A continuación se da la resolución completa y prolija, separada en (a) y (b), verificando cada paso.

(A) ALGORITMO

Cada competidor i tiene tiempo total de natación s_i , y tiempo de “cola” $b_i + r_i$ (lo que le queda por hacer después de salir de la piscina, que puede solaparse libremente con otros). Como la piscina es el único recurso exclusivo, el orden de largada determina en qué instante cada competidor **sale** de la piscina (que es simplemente la suma acumulada de los s_i de los competidores que largaron antes, incluido él mismo), y desde ahí cada uno corre su $b_i + r_i$ de forma independiente.

```
OrdenTriatlon(competidores):
    ordenar los competidores por (b_i + r_i) DECRECIENTE
    devolver ese orden como orden de largada
```

Complejidad: $O(n \log n)$ por el ordenamiento.

Por qué “decreciente en $b_i + r_i$ ” y no en s_i : intuitivamente, el competidor que más tiempo va a necesitar *después* de salir de la piscina (mayor $b_i + r_i$) debe entrar a nadar lo antes posible, para no arrancar tarde su etapa “larga”; los competidores con poco trabajo restante ($b_i + r_i$ chico)

pueden esperar más en la cola de la piscina sin perjudicar el tiempo total, porque terminan rápido una vez que salen.

(B) CORRECCIÓN — ARGUMENTO DE INTERCAMBIO

Sea $T = (i_1, \dots, i_n)$ el orden que produce el algoritmo (por $b_i + r_i$ decreciente) y sea $T^* = (j_1, \dots, j_n)$ un orden óptimo arbitrario.

Definición de inversión. Decimos que T^* tiene una inversión si hay dos competidores consecutivos $j_k = i$, $j_{k+1} = j$ (i nada inmediatamente antes que j) tales que $b_i + r_i < b_j + r_j$ (el que nada después tiene *más* trabajo restante que el que nada antes — contradice el orden ávido).

El intercambio no empeora el tiempo total. Supongamos que T^* tiene una inversión entre el par adyacente (i, j) con $b_i + r_i < b_j + r_j$. Sea t el instante en que i **empieza** a nadar en T^* (todos los competidores antes del par no se ven afectados por el intercambio, ya que el par sigue ocupando el mismo intervalo total de la piscina). En T^* , i nada en $[t, t + s_i]$ y j nada en $[t + s_i, t + s_i + s_j]$; i sale de la piscina en $t + s_i$ y termina toda su prueba en $t + s_i + b_i + r_i$; j sale en $t + s_i + s_j$ y termina en $t + s_i + s_j + b_j + r_j$.

Construimos T' intercambiando i y j (j nada primero): j nada en $[t, t + s_j]$ y termina en $t + s_j + b_j + r_j$; i nada en $[t + s_j, t + s_j + s_i]$ y termina en $t + s_j + s_i + b_i + r_i$. Observar que $t + s_j + s_i = t + s_i + s_j$ (el instante en que el segundo del par sale de la piscina es el mismo en ambos órdenes, ya que la piscina se ocupa $s_i + s_j$ en total de cualquier manera). Por lo tanto:

- El tiempo de finalización de i en T' es $t + s_j + s_i + b_i + r_i$, el mismo que el de j en T^* ($t + s_i + s_j + b_j + r_j$) pero con $b_i + r_i$ en vez de $b_j + r_j$ — y como $b_i + r_i < b_j + r_j$, el nuevo tiempo de fin de i en T' es **estrictamente menor** que el tiempo de fin de j en T^* .
- El tiempo de finalización de j en T' es $t + s_j + b_j + r_j$ — comparado con el tiempo de fin de i en T^* ($t + s_i + b_i + r_i$), no hay una relación directa simple, pero lo relevante es el **máximo** de los dos tiempos de fin del par, ya que eso es lo que puede afectar el tiempo total de la competencia.

El máximo de los dos tiempos de fin en T^* es $\max(t + s_i + b_i + r_i, t + s_i + s_j + b_j + r_j) = t + s_i + s_j + b_j + r_j$ (el segundo, j , termina después porque nada al final y además tiene más trabajo restante — de hecho el máximo del par en T^* SIEMPRE lo alcanza j , el que nada después, dado que termina en un instante estrictamente posterior al de i). El máximo de los dos tiempos de fin en T' es $\max(t + s_j + b_j + r_j, t + s_j + s_i + b_i + r_i)$ — acotando el segundo término: $t + s_j + s_i + b_i + r_i < t + s_i + s_j + b_j + r_j$ (por la hipótesis $b_i + r_i < b_j + r_j$) — es decir, el máximo del par en T' es a lo sumo $t + s_i + s_j + b_j + r_j$, el mismo valor que era el máximo en T^* . Por lo tanto, **el máximo tiempo de finalización del par no aumenta**, y como los demás competidores no se ven afectados, **el tiempo total de la competencia en T' es $\leq$ al de T^* .**

Iterar. Cada intercambio de un par adyacente invertido elimina esa inversión sin crear inversiones nuevas entre otros pares, y el número de inversiones es finito y decrece estrictamente. Iterando se llega, sin empeorar el tiempo total, a un orden sin inversiones — es decir, ordenado por $b_i + r_i$ decreciente, exactamente el orden T que produce el algoritmo. Por lo tanto **T es óptimo.** ■

Ejemplo numérico de verificación. 3 competidores: $X(s=2, b=3, r=4 \rightarrow b+r=7)$, $Y(s=3, b=1, r=1 \rightarrow b+r=2)$, $Z(s=1, b=5, r=5 \rightarrow b+r=10)$.

Orden ávido ($b+r$ decreciente): $Z(10)$, $X(7)$, $Y(2)$. Traza: Z nada $[0, 1]$, sale a $t=1$, termina en $1+10=11$. X nada $[1, 3]$, sale a $t=3$, termina en $3+7=10$. Y nada $[3, 6]$, sale a $t=6$, termina en $6+2=8$. Tiempo total = $\max(11, 10, 8) = 11$.

Comparando con el orden inverso (Y, X, Z): Y nada $[0, 3] \rightarrow$ termina $3+2=5$; X nada $[3, 5] \rightarrow$ termina $5+7=12$; Z nada $[5, 6] \rightarrow$ termina $6+10=16$. Tiempo total=**16**, peor que 11 ✓ (consistente con que el orden ávido es mejor).

Ejercicio 3 (K&T Ex. 4.18) — Camino más rápido con tiempos de viaje variables

Enunciado. Grafo dirigido $G=(V, E)$, $n=|V|$, $m=|E|$. Cada arista (u, v) tiene una función $f_{uv}(t) =$ tiempo de llegada a v si se sale de u en el instante t , con $f_{uv}(t) \geq t$ (no se retrocede en el tiempo) y f_{uv} no decreciente en t . Se calcula $f_{uv}(t)$ en $O(1)$. Encontrar la forma más rápida de llegar de o a d saliendo de o en $t=0$, con un algoritmo $O(m \log n)$.

(A) ALGORITMO — VARIANTE DE DIJKSTRA

La idea es exactamente la misma que Dijkstra, cambiando “sumar el peso de la arista” por “aplicar la función f_e al tiempo actual”:


```

CaminoMasRapido(G, o):
    S = ∅                                # nodos con tiempo de llegada ya determinado (correcto)
    d(o) = 0; r(o) = null                 # r(v): nodo anterior a v en el camino más rápido
    d(v) = ∞ para v ≠ o
    Q = cola de prioridad con todos los vértices, clave d(·)
    mientras Q no vacía:
        u = extraer_mín(Q)               # el de menor d(u) entre los no procesados
        S = S ∪ {u}
        para cada arista (u,v) ∈ E:
            si f_(u,v)(d(u)) < d(v):
                d(v) = f_(u,v)(d(u))
                r(v) = u
            actualizar la clave de v en Q
    devolver d(d) y reconstruir el camino siguiendo r(·) desde d hasta o

```

(B) CORRECCIÓN

Invariante a probar por inducción en $|S|$: en cualquier momento del algoritmo, para todo $v \in S$, $d(v)$ es efectivamente el instante más temprano posible en el que se puede llegar a v desde o .

- **Caso base ($|S|=1$):** $S=\{o\}$, $d(o)=0$, que es trivialmente el instante más temprano de llegar a o (no se puede llegar antes de salir).
- **Paso inductivo:** supongamos que la propiedad vale para $|S|=k$, y el algoritmo agrega el nodo v (el de menor $d'(v) = \min$ sobre aristas (u,v) con $u \in S$ de $f_{(u,v)}(d(u))$). Sea (u,v) la arista que logra ese mínimo — por hipótesis inductiva, $d(u)$ es el tiempo real más temprano de llegar a u , así que $d(v)=f_{(u,v)}(d(u))$ es el tiempo de un camino real y válido hasta v (llegar a u en $d(u)$, y de ahí viajar a v). Falta ver que **ningún** camino puede llegar a v más temprano.

Sea P cualquier camino de o a v . P debe salir de S en algún punto: sea y el primer nodo de P que no pertenece a S , y sea $x \in S$ el nodo justo antes de y en P (x puede ser o mismo si $o \notin S$... pero $o \in S$ siempre). Como f_{xy} es no decreciente y no retrocede en el tiempo, el instante en que P llega a y es $\geq$ el instante en que llega a x , que por hipótesis inductiva es $\geq d(x)$. Por definición del algoritmo (v es el nodo de mínimo $d'(\cdot)$ entre TODOS los candidatos con un extremo en S , incluyendo el candidato “llegar a y vía x ”), el tiempo de llegar a y por este camino es $\geq d(v)$ (porque v fue elegido como el mínimo entre todos esos candidatos, y “llegar a y vía (x,y) ” es uno de ellos, o v mismo es y). Como f_e nunca retrocede en el tiempo para el resto del camino P desde y hasta v (si $y \neq v$), el tiempo final de P es $\geq$ tiempo de llegar a y $\geq d(v)$. Por lo tanto **ningún** camino llega a v antes que $d(v)$. ■

(C) COMPLEJIDAD $O(M \log N)$

Usando una cola de prioridad (heap binario) igual que en la implementación estándar de Dijkstra: cada nodo se extrae del heap una única vez ($O(n \log n)$ en total por las n extracciones), y cada arista se examina exactamente una vez, generando a lo sumo una actualización de clave en el heap ($O(\log n)$ cada una, ya que el cálculo de $f_e(t)$ es $O(1)$ por hipótesis) — $O(m \log n)$ en total por las actualizaciones. Sumando: $O(n \log n + m \log n) = O(m \log n)$ (ya que el grafo es conexo desde 0, $m \geq n-1$, así que $n \log n = O(m \log n)$). ■

Práctico 6

Ejercicio 1 (K&T Ex. 4.8) — Unicidad del MST con costos distintos

Enunciado. Mostrar que un grafo conexo con costos de arista todos distintos tiene un único árbol de expansión mínima.

Demostración por absurdo, usando la propiedad de corte. Supongamos que existen dos MST distintos T y T' de G , ambos de costo mínimo $c(T)=c(T')$. Como $T \neq T'$, existe alguna arista $e' \in T' \setminus T$. Al agregar e' a T se forma un ciclo C (agregar cualquier arista a un árbol de expansión crea exactamente un ciclo). Sea e la arista de **mayor** costo en C .

Como todos los costos son distintos, C tiene una arista de mayor costo bien definida, e . Por la propiedad del ciclo (enunciado 4.20 del libro): dado un ciclo C y e la arista de mayor costo en C , e **no** pertenece a ningún MST de G . Pero $e \in C \subseteq T \cup \{e'\}$, y como e es la arista de mayor costo del ciclo, y sabemos que $e' \in T'$ (por construcción) y las demás aristas de C están en T (porque C =camino en T entre los extremos de e' , más e'), hay dos casos:

- Si $e = e'$: entonces e' es la arista de mayor costo del ciclo, y por la propiedad del ciclo e' no puede pertenecer a ningún MST — pero $e' \in T'$, contradicción (T' es MST).
- Si $e \neq e'$: entonces e es una arista de T (ya que $C \setminus \{e'\} \subseteq T$), y por la propiedad del ciclo e no puede pertenecer a ningún MST — pero $e \in T$, contradicción (T es MST).

En ambos casos se llega a un absurdo. Por lo tanto, no pueden existir dos MST distintos: **el MST es único.** ■

⚠ Nota sobre Semana 6, Ej.1: la resolución oficial hace esencialmente el mismo argumento (con T' y T como los dos MST hipotéticos y usando la propiedad del ciclo), pero no distingue explícitamente los dos casos ($e=e'$ vs $e \neq e'$) — da por hecho, sin decirlo, que “ e ” (la arista más cara del ciclo) es “la nueva arista”, lo cual no es necesariamente cierto: la arista de mayor costo en el ciclo puede ser cualquiera de las que lo componen, no necesariamente e' . La demostración de arriba completa ese detalle explícitamente; la conclusión final del documento de cátedra es correcta.

Ejercicio 2 (K&T Ex. 4.21) — MST de un “casi árbol” en tiempo $O(n)$

Enunciado. Un “casi árbol” es un grafo conexo con n vértices y m aristas tal que $m \leq n+8$. Dar un algoritmo $O(n)$ para construir su MST, demostrar corrección y complejidad.

(A) ALGORITMO

Como un árbol de expansión tiene exactamente $n-1$ aristas, un casi árbol tiene a lo sumo $(n+8)-(n-1) = 9$ aristas “de más” (aristas cuya eliminación, una por una, mantiene el grafo conexo).

```
MST_CasiArbol(G):
    H = G
    repetir hasta 9 veces (o hasta que  $|E(H)| = n-1$ , lo que ocurra antes):
        ejecutar BFS/DFS desde cualquier nodo de H y detectar un ciclo
        (si no se encuentra ningún ciclo, H ya es un árbol: detener)
        sea e la arista de mayor costo en ese ciclo
         $H = H \setminus \{e\}$ 
    devolver H
```

Cómo detectar un ciclo en $O(|V(H)|+|E(H)|)$: correr DFS llevando un vector de padres; la primera arista (u,v) que se examina hacia un nodo v ya visitado y que no es el padre directo de u cierra un ciclo — ese ciclo se reconstruye siguiendo los punteros de padre desde u hasta v , y agregando la arista (u,v) (técnica ya usada en el Ejercicio 4 del Práctico 3 de Grafos).

(B) CORRECCIÓN

Cada iteración aplica la **propiedad del ciclo** (enunciado 4.20): dado el ciclo C encontrado y e la arista de mayor costo de C , e no pertenece a ningún MST de H . Por lo tanto, eliminar e de H no cambia el MST: cualquier MST de H (antes de eliminar e) sigue siendo un MST del grafo $H \setminus \{e\}$ (ya que ningún MST usaba e , y $H \setminus \{e\}$ sigue conteniendo todas las aristas relevantes; además $H \setminus \{e\}$ sigue siendo conexo, porque eliminar una arista de un ciclo nunca desconecta un grafo — las otras aristas del ciclo siguen dando un camino alternativo entre sus extremos).

Repitiendo este proceso, en cada paso se elimina una arista “sobrante” (que forma parte de un ciclo) sin afectar el MST y sin desconectar el grafo. Como el grafo original tiene a lo sumo 9 aristas de más respecto de un árbol ($m-(n-1) \leq 9$), después de a lo sumo 9 eliminaciones el grafo resultante H tiene exactamente $n-1$ aristas y sigue siendo conexo — es decir, **H es un árbol de expansión**, y por construcción (cada arista eliminada no podía pertenecer a ningún MST) **H es el MST de G**. ■

(C) COMPLEJIDAD $O(N)$

Cada iteración hace una detección de ciclo por DFS/BFS, que cuesta $O(|V(H)| + |E(H)|) = O(n+m)$. Como $m \leq n+8 = O(n)$, cada iteración es $O(n)$. Se realizan a lo sumo **9 iteraciones** (constante, no depende de n), así que el costo total es $9 \cdot O(n) = O(n)$. ■

⚠ **Precisión sobre Semana 6, Ej.2:** la resolución oficial es correcta en su núcleo (aplicar la propiedad del ciclo hasta 9 veces, complejidad $O(n)$ porque $m=O(n)$), pero no explicita **por qué** eliminar una arista de un ciclo nunca desconecta el grafo (paso necesario para que “H siga siendo conexo” en cada iteración, que se usa implícitamente). Se agregó esa justificación arriba para completar el argumento de corrección.

Ejercicio 3 (K&T Ex. 4.10) — MST tras agregar una arista

Enunciado. $G=(V,E)$, n vértices, costos c_e todos distintos. T es un MST de G . Se agrega una arista e a G , obteniendo G' . Sea $T \cup \{e\}$ el grafo resultante de agregar e a T ; como T es árbol y e es una arista nueva, $T \cup \{e\}$ tiene exactamente un ciclo (el que se cierra al conectar los dos extremos de e , que ya estaban conectados por el único camino que existe entre ellos en el árbol T). Sea $e_{\max}$ la arista de mayor costo de ese ciclo, $c_{\max}$ su costo, y $T' = (T \cup \{e\}) \setminus \{e_{\max}\}$.

(A) MOSTRAR QUE T' ES UN MST DE G'

i) **T' es un árbol de expansión de G' .** $T \cup \{e\}$ tiene $n-1+1 = n$ aristas y exactamente un ciclo (el generado por e). Quitar cualquier arista de ese ciclo —en particular $e_{\max}$ — rompe el ciclo sin desconectar el grafo (mismo argumento que en el Ejercicio 2: las demás aristas del ciclo siguen dando un camino alternativo entre los extremos de la arista quitada), dejando T' con $n-1$ aristas y conexo, tocando los n vértices. Por lo tanto (V, T') es un árbol de expansión de G' .

ii) **Ninguna arista fuera de T' puede pertenecer a un MST de G' .** Las aristas que no están en T' son exactamente $(E \setminus T) \cup \{e_{\max}\}$ (todas las aristas de G que no estaban en T originalmente, más $e_{\max}$ que se acaba de sacar). Hay que distinguir dos grupos:

- **$e_{\max}$:** por definición, es la arista de mayor costo del ciclo que se forma en $T \cup \{e\}$. Por la propiedad del ciclo (4.20) aplicada a ese ciclo en G' , $e_{\max}$ no pertenece a ningún MST de G' .

- **Cada arista $f \in E \setminus T$ (arista de G que no estaba en el MST original T):** como T es MST de G , por la propiedad del ciclo aplicada al ciclo que se forma al agregar f a T (dentro de G , sin considerar e todavía), f es la arista de mayor costo de ese ciclo (de lo contrario T no sería MST de G). Ese mismo ciclo existe también en G' ($T \subseteq G'$), así que f sigue siendo la arista de mayor costo de un ciclo de G' — por la propiedad del ciclo, f tampoco pertenece a ningún MST de G' .

Como ninguna arista fuera de T' puede estar en un MST de G' , y T' es un árbol de expansión válido de G' con $n-1$ aristas, se concluye que **T' debe ser exactamente el MST de G'** (todo MST de G' tiene $n-1$ aristas, ninguna de las cuales puede estar fuera de T' por lo anterior — y como $|T'|=n-1$, el MST coincide con T' arista por arista). ■

⚠ **Aclaración sobre la resolución “oficial” de este ejercicio (Semana 5, Ej.3, y también el enfoque de Practico 6 2024):** el documento de cátedra desarrolla una prueba alternativa (comparando tres ciclos C , C' , K) que es la del propio libro de Kleinberg & Tardos, y es correcta, pero considerablemente más indirecta que el argumento por la propiedad del ciclo mostrado arriba (que evita tener que introducir el ciclo K y razonar sobre tres ciclos simultáneos). Ambos caminos llegan a la misma conclusión válida; se optó por presentar la versión más directa.

(B) ALGORITMO $O(N)$ PARA CONSTRUIR EL MST DE G' A PARTIR DE G , T Y E

```

ActualizarMST( $T$ ,  $e=(v,w)$ ):
    encontrar el camino  $P$  entre  $v$  y  $w$  dentro del árbol  $T$            #  $O(n)$ , recorriendo  $T$ 
     $e_{\max}$  = arista de mayor costo en  $P$                              #  $O(n)$ , un recorrido de  $P$ 
    si  $\text{costo}(e) < \text{costo}(e_{\max})$ :
         $T' = (T \setminus \{e_{\max}\}) \cup \{e\}$ 
    si no:
         $T' = T$                                                        #  $T$  ya era MST de  $G'$ 
    devolver  $T'$ 
  
```

Corrección: si $\text{costo}(e) \geq \text{costo}(e_{\max})$, entonces al agregar e a T el ciclo que se forma tiene a $e_{\max}$ (o eventualmente a e misma, si e es la más cara) como arista de mayor costo — en cualquier caso, por la parte (a), la arista de mayor costo del ciclo $T \cup \{e\}$ no pertenece a ningún MST de G' ; si esa arista es e misma ($\text{costo}(e) \geq \text{costo}(e_{\max})$), entonces T (que no contiene a e) sigue siendo un MST válido de G' , porque T ya era MST de G y $G'=G \cup \{e\}$ con e no perteneciente a ningún MST de G' . Si $\text{costo}(e) < \text{costo}(e_{\max})$, por la parte (a) $T' = (T \cup \{e\}) \setminus \{e_{\max}\}$ es el MST de G' .

Complejidad: encontrar el camino entre v y w en el árbol T se hace con una búsqueda (BFS/DFS) restringida al árbol, que tiene $n-1$ aristas — $O(n)$. Encontrar la arista de mayor costo en ese camino es $O(\text{longitud del camino}) = O(n)$. El resto de las operaciones son $O(1)$. Total: **$O(n)$** . ■

Ejemplo numérico de verificación. Retomando el grafo de la sección 5 de la guía temática (A,B,C,D,E,F con MST de costo 13 = {B–C(1), A–C(2), D–E(2), E–F(3), B–D(5)}), agreguemos la arista nueva $e=(A,F)$ con costo 3.

Camino entre A y F en T : A–C(2)–B(1)–D(5)–E(2)–F(3) → costo máximo en el camino es 5 (arista B–D). Como $\text{costo}(e)=3 < 5$, se reemplaza: $T' = T \setminus \{B-D(5)\} \cup \{A-F(3)\}$.

$T' = \{B-C(1), A-C(2), D-E(2), E-F(3), A-F(3)\}$, costo total = $1+2+2+3+3 = 11$ (mejora respecto del costo 13 anterior, como se espera al agregar una arista más barata que rompe el “cuello de botella” B–D). Verificado con Kruskal desde cero sobre el grafo con la arista extra: el MST resultante tiene efectivamente costo 11 y coincide con T' arista por arista. ✓

Resumen de técnicas usadas en estos prácticos

Ejercicio	Técnica
P5-Ej.1 (estaciones)	Greedy stays ahead sobre la posición de la k -ésima estación
P5-Ej.2 (triatlón)	Exchange argument sobre pares adyacentes (inversiones en b_i+r_i)
P5-Ej.3 (viaje variable)	Variante de Dijkstra; invariante de corrección análogo al de caminos mínimos
P6-Ej.1 (unicidad MST)	Propiedad del ciclo aplicada por absurdo a dos MST hipotéticos
P6-Ej.2 (casi árbol)	Propiedad del ciclo aplicada repetidamente (a lo sumo 9 veces)
P6-Ej.3 (MST + arista nueva)	Propiedad del ciclo aplicada al ciclo formado por la nueva arista

Para el examen: los Ejercicios 1 y 3 del Práctico 6 muestran el patrón más reutilizable de todo el módulo de MST: casi cualquier pregunta de “¿sigue siendo válido el MST si cambio algo del grafo (agrego una arista, quito una arista, cambio un costo)?” se resuelve identificando el o los ciclos relevantes y aplicando la **propiedad del ciclo** (4.20) para decidir qué arista no puede estar en ningún MST. Es el mismo rol que cumplía la “reducción a un problema ya resuelto” en el

práctico de grafos: acá el problema base es siempre “propiedad del ciclo + propiedad del corte”, y el trabajo real es identificar correctamente el ciclo o el corte relevante en cada variante del problema.